天津大學

《机器学习与深度学习》实验报告

LeNet-5 模型在 Cifar-10 数据集的改进与优化

学　　号_____

姓　　名_____

邮　　箱____

学　　院_____智算学部_____

专　　业_____人工智能_____

年　　级_____23_____

任课教师_____王煜、曹兵_____

2025 年 5 月 20 日

目　录

一、引言.....1
1.1　实验背景介绍.....1
1.2　实验目标与内容.....1
二、模型改进.....3
2.1　使用小尺寸卷积核，加深层数.....3
2.2　VGG 堆叠.....4
2.3　引入残差.....8
2.4　稠密连接.....10
2.5　网中网.....12
三、总结与体会.....16

一、引言

1.1 实验背景介绍

LeNet 是由 AT&T 贝尔实验室一个以杨立昆（Yann LeCun）为核心的研究团队在 1988 年至 1998 年期间创建的一系列卷积神经网络架构，旨在读取手写数字和字母的小型灰度图像，并应用于自动柜员机的支票识别。

卷积神经网络是一种前馈神经网络，其人工神经元能够对覆盖范围内周围细胞的一部分做出响应，在大规模图像处理中表现出色。LeNet-5 是最早的卷积神经网络之一，在深度学习的发展历程中具有重要的历史地位。

1.2 实验目标与内容

LeNet-5 模型包含了 7 层网络，分别为：输入层，卷积层 C1，池化层 S2，卷积层 C3，池化层 S4，全连接层 C5，全连接层，输出层，由图 1-1 所示：

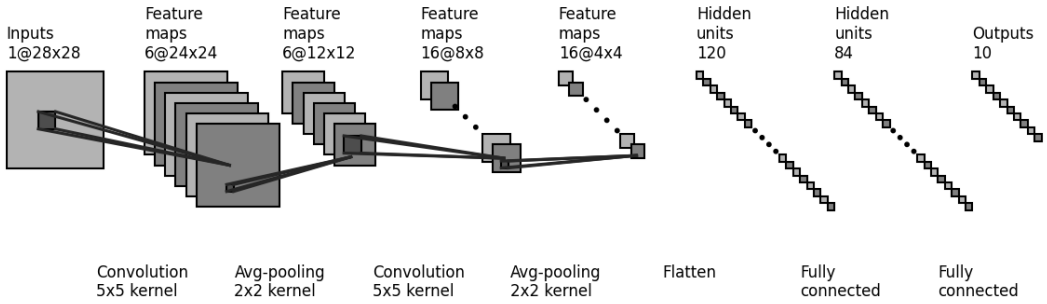

图 1-1　LeNet-5 结构

该模型在手写数字 MNIST 数据集中表现极佳，性能参数由图 1-2 所示，测试集准确率可达 99%，基本可以认为都能识别成功。

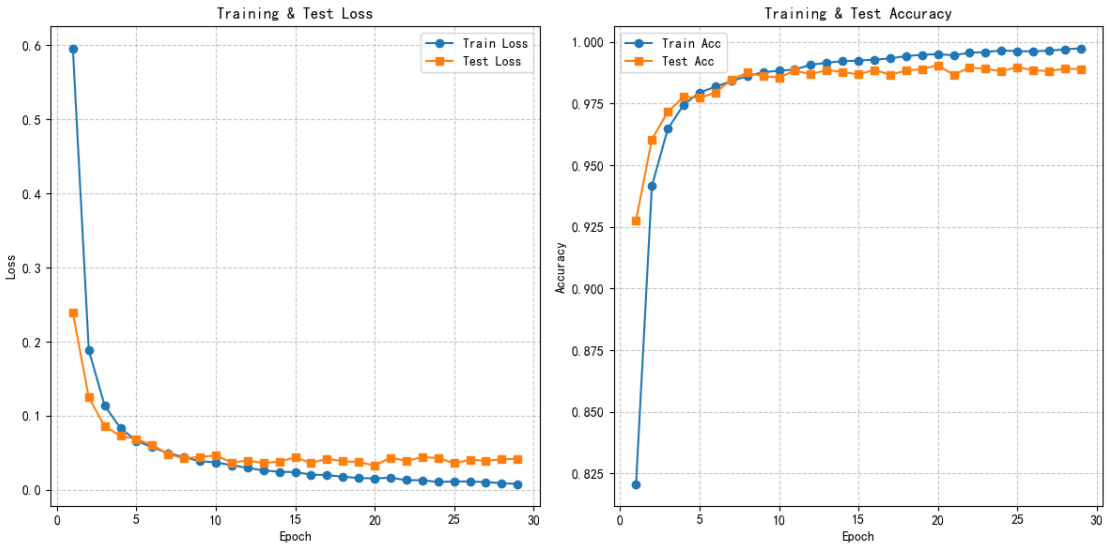

图 1-2　LeNet 在 MNIST 数据集的表现

然而，如果数据集换成更加复杂的 Cifar-10 数据集，那么表现则不尽如人意，由图 1-3 所示，准确率最高才能达到 70%。

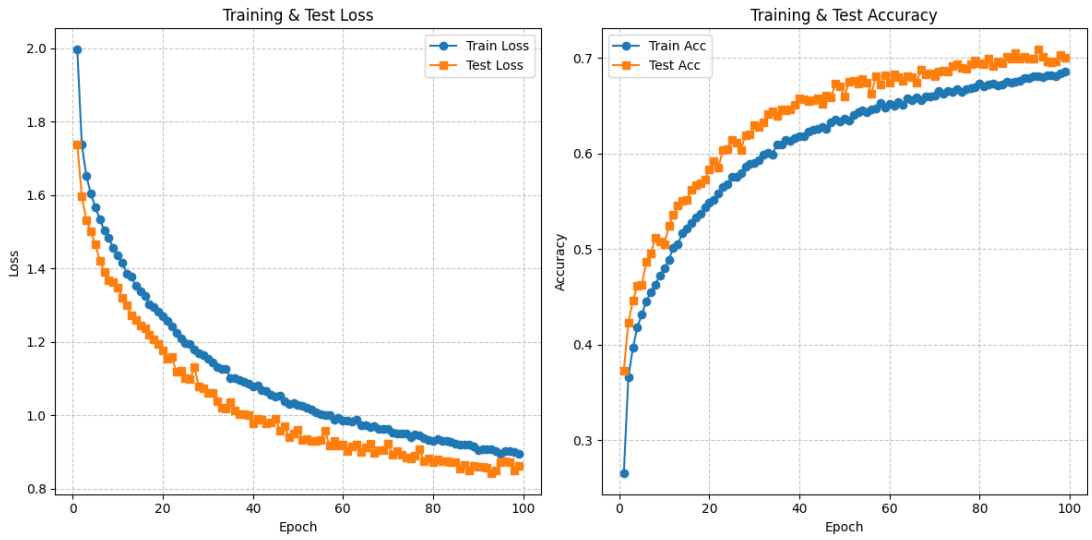

图 1-3　LeNet 在 Cifar-10 数据集的表现

这是因为在数据复杂度方面，MNIST 更加简单，仅是单通道的灰度图，而 CIFAR-10 包含了多通道颜色信息、复杂物体结构和背景干扰，需要更深的网络和更强的特征提取能力，而 LeNet-5 仅有约 6 万参数，浅层的网络无法学习复杂的高层次特征，在进行训练时，也没有使用一些正则化的技术。

因此，在本次实验中，将使用一些方法，包括加深层数，改变池化层，改变卷积核等，来对 LeNet 模型进行改进，以更加适应 Cifar-10 数据集。

二、模型改进

2.1 使用小尺寸卷积核，加深层数

由于使用多个小尺寸卷积核与使用大尺寸卷积核可以获得相同的感受野，并且使用多个小尺寸卷积核可以使用到更多次数的激活函数，因此可以获得更多的拟合能力，此外，小卷积核还能显著减少参数量，例如，两个 3×3 卷积层的叠加等效于一个 5×5 卷积层的感受野，但参数量从 25 减少到 2×9=18。因此，可以考虑首先减少卷积核尺寸，并增加网络深度。

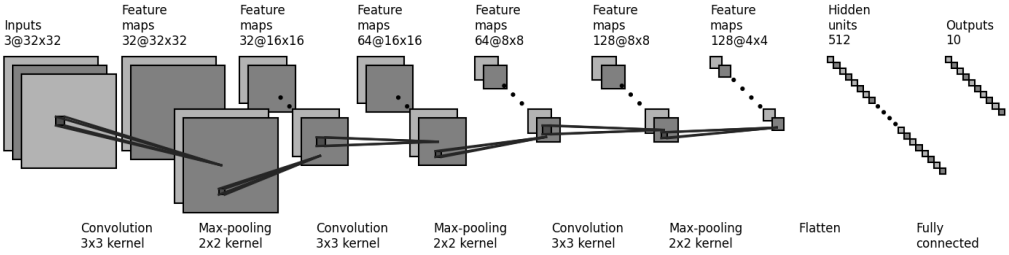

图 2-1　LeNetV1 结构

由图 2-1 所示，与 LeNet 相比，使用 3*3 的卷积核，并额外增添了一个卷积层。

平均池化表示取范围内的平均值，可以保留图像的整体分布，而最大池化则在范围内取最大值，这样可以突出图像的显著特征，例如物体边缘，角点，这些特征在分类任务中更具判别性，因此，在本次模型改进中，使用最大池化。

批量归一化是一种通过对神经网络层输入进行标准化处理，加速训练并提升模型稳定性的技术，使用该技术具有加速训练收敛，降低模型对参数初始化的敏感度，减少过拟合风险等优势，因此，本次改进也使用了批量归一化。

以及在全连接层添加 Dropout 正则化，使 50%的神经元失效，以防止全连接层的过拟合。

代码实现为：

```python
class LeNet5_v1(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool3 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(128*4*4, 512)  # 全连接层
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, 10)  # 输出层
    def forward(self, x):
        x = self.pool1(F.relu(self.bn1(self.conv1(x))))
        x = self.pool2(F.relu(self.bn2(self.conv2(x))))
        x = self.pool3(F.relu(self.bn3(self.conv3(x))))
        x = x.view(x.size(0), -1)
        x = self.dropout(F.relu(self.fc1(x)))
        return self.fc2(x)
```

图 2-14　LeNetV1 代码实现

使用这样的模型进行训练测试，得到的结果如图 2-2 所示：

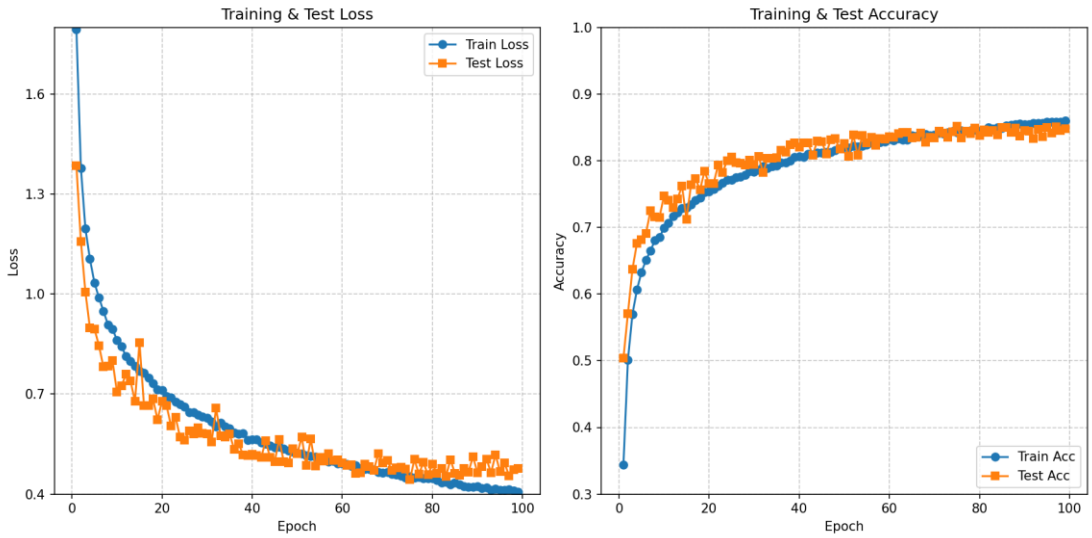

图 2-2　LeNetV1 模型表现

训练损失和测试损失，训练准确率和测试准确率最终都达到较优水平，且二者差距较小，说明模型没有出现严重的过拟合现象，具备良好的泛化能力。

改进型的模型正确率和损失函数得到了很好的提升，准确率最高可达 83%，与改进之前相比提升了 13%左右。

2.2　VGG 堆叠

VGG-16 是由牛津大学 Visual Geometry Group（VGG）在 2014 年提出的经典卷积神经网络模型，核心思想是通过堆叠小尺寸卷积核和增加网络深度来提升特征提取能力，成为后续深度学习的重要基础。

VGG-16 包含 13 个卷积层和 3 个全连接层，共 16 层，如图 2-3 所示：

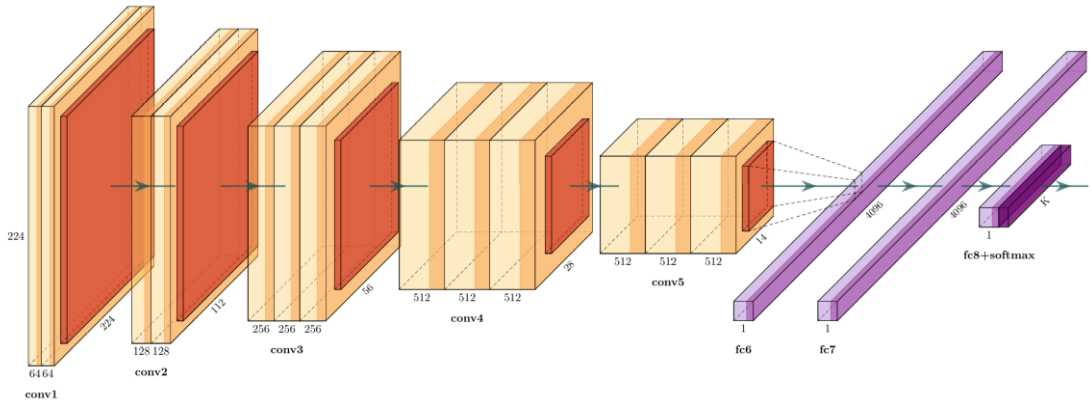

图 2-3 VGG16 模型

模型的卷积阶段组织成一个个 block，通过多个 block 的堆叠来提高深度，因此，也可以采用相似的思想，对模型进行改进。

改进后的 LeNet_V2 版本结构如图 2-4 所示，使用小卷积核，稍少的 block 数，以适配 Cifar-10 数据集的特点。

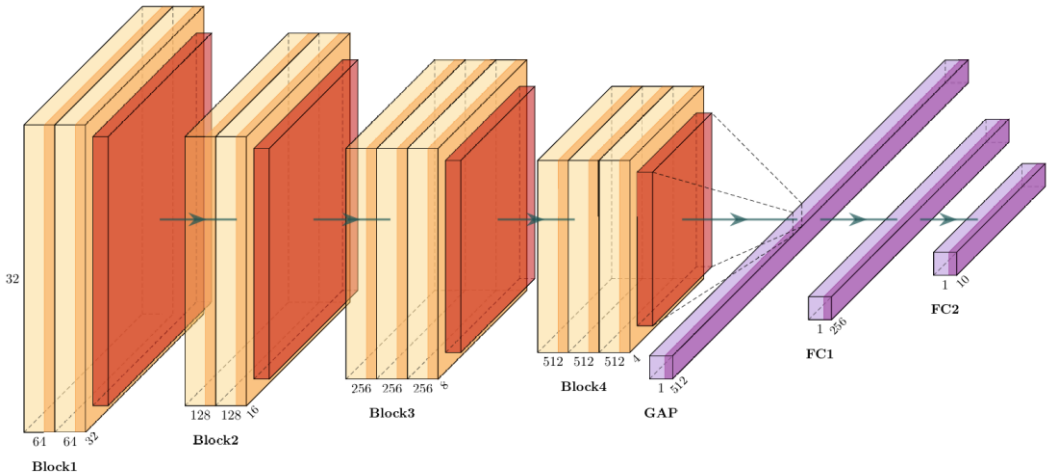

图 2-4　LeNet_v2 模型

首先定义出一个 block：

```
# Block 1 (2x conv)
nn.Conv2d(3, 64, kernel_size=3, padding=1),  # 32x32
nn.BatchNorm2d(64),
nn.ReLU(inplace=True),
nn.Conv2d(64, 64, kernel_size=3, padding=1),
nn.BatchNorm2d(64),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),  # 16x16
```

图 2-15　首个 block 定义

然后使用这样的 block，重复 4 次，并在 block 内部修改通道数。

```python
# 特征提取层
self.features = nn.Sequential(
    # Block 1 (2x conv)
    nn.Conv2d(3, 64, kernel_size=3, padding=1),  # 32x32
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.Conv2d(64, 64, kernel_size=3, padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),  # 16x16
    # Block 2 (2x conv)
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(inplace=True),
    nn.Conv2d(128, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),  # 8x8
    # Block 3 (3x conv)
    nn.Conv2d(128, 256, kernel_size=3, padding=1),
    nn.BatchNorm2d(256),
    nn.ReLU(inplace=True),
    nn.Conv2d(256, 256, kernel_size=3, padding=1),
    nn.BatchNorm2d(256),
    nn.ReLU(inplace=True),
    nn.Conv2d(256, 256, kernel_size=3, padding=1),
    nn.BatchNorm2d(256),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),  # 4x4
    # Block 4 (3x conv)
    nn.Conv2d(256, 512, kernel_size=3, padding=1),
    nn.BatchNorm2d(512),
    nn.ReLU(inplace=True),
    nn.Conv2d(512, 512, kernel_size=3, padding=1),
    nn.BatchNorm2d(512),
    nn.ReLU(inplace=True),
    nn.Conv2d(512, 512, kernel_size=3, padding=1),
    nn.BatchNorm2d(512),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(kernel_size=2, stride=2),  # 2x2
)
```

图 2-16　LeNetV2 特征提取定义

使用这样的模型进行训练，得到的结果如图 2-5 所示：

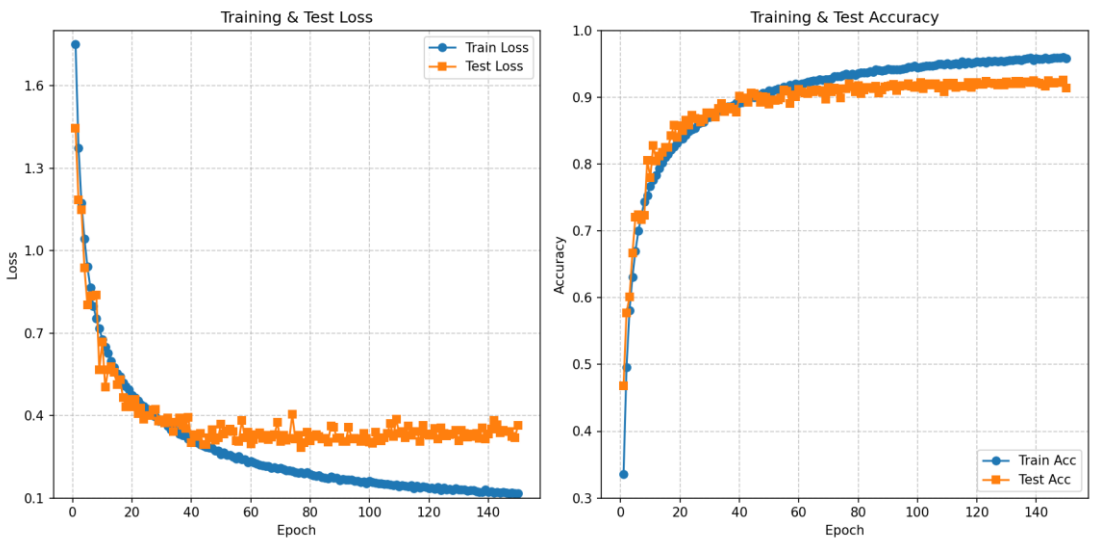

图 2-4　LeNetV2 表现

准确率和损失与 V1 相比，有较大提升，约为 7%，随着训练轮数增加，发现测试集与训练集的差别越来越大，表明出现了一定程度的过拟合。

在此基础上，如果再次增加层数，那么将不会有一些优化的效果，例如，如果只是增加 1 个 block，共计 5 个 block，得到的网络结构如下：

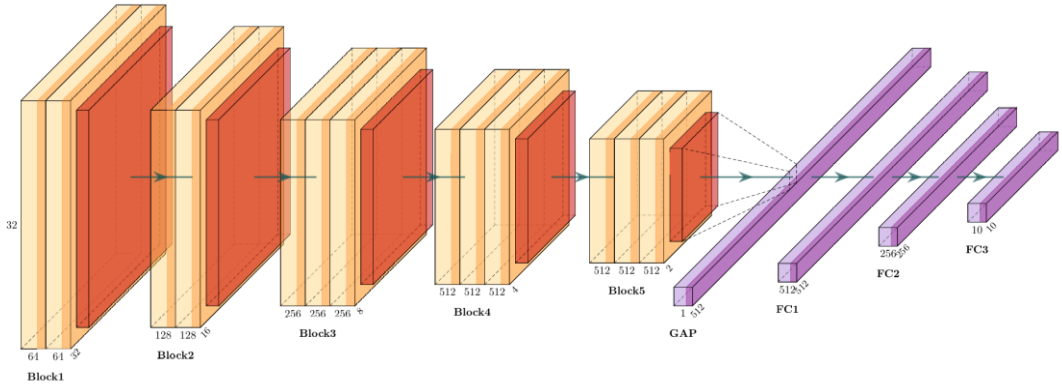

图 2-5　LeNetV2_p1 结构

新增的模型定义为：

```
# Block 5
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Conv2d(512, 512, kernel_size=3, padding=1),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),  # 1x1
```

图 2-17　新增 block 定义

使用该模型进行训练，得到的结果如下：

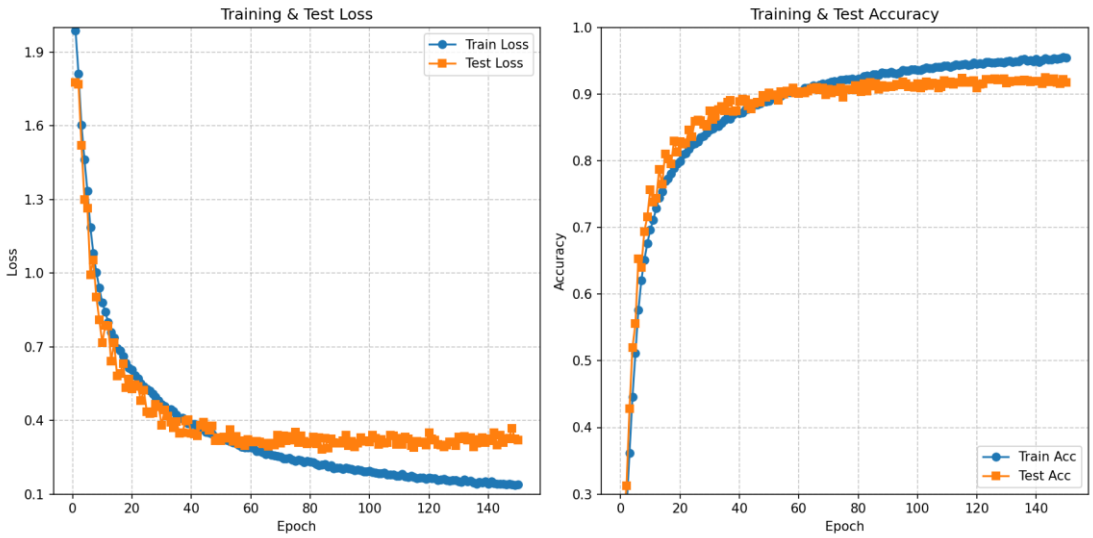

图 2-6　LeNetV2_p1 表现

此时模型的准确率最高可达 92%，与仅有 4block 的模型相比，并无差异。

这是因为网络退化问题，当网络层数增加到一定程度时，训练误差可能反而会升高，即使通过 BatchNorm、ReLU 等方法解决了一定程度梯度消失/爆炸问题，但是该现象依然存在，且如果层数过高，梯度消失也会出现。

2.3 引入残差

2015 年，何恺明等人提出残差网络（ResNet），在 ImageNet 分类任务中首次训练出 152 层网络，错误率大幅降低。

残差网络通过残差连接，引入残差学习，将网络层的学习目标从直接映射改为学习残差，可以有效缓解 2.2 中的问题。

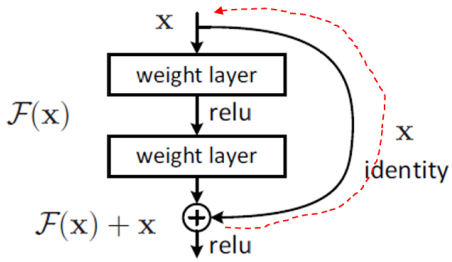

图 2-7　残差块

由图 2-7 所示，残差学习通过跳跃连接，提供一条无参数、无非线性变换的路径，梯度可直接反向传播至浅层，避免因多次连乘导致梯度消失。

此时学习的目标就从拟合直接映射$\mathcal{F}(x)$变为了拟合出残差映射$\mathcal{F}(x) - x$。残差映射往往更容易优化。

残差网络的一个经典案例是 ResNet18，它的一个残差块的构造如下图 2-8，当输入和输出的维度不匹配时，采用 1x1 卷积来调整维度。

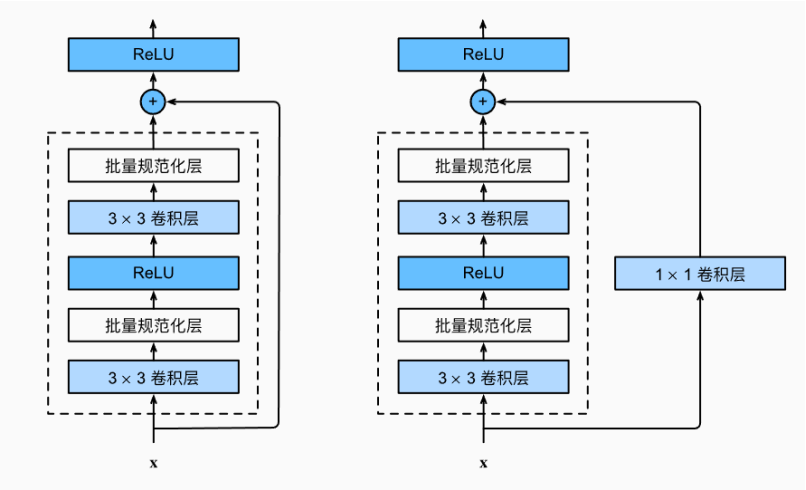

图 2-8　带有 1x1 卷积的和不带有的残差块

我们将使用这个残差块来优化 LeNetV3 版本，网络结构如图 2-9 所示：

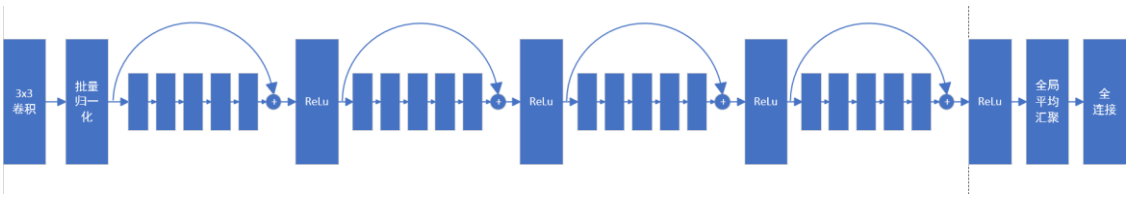

图 2-9　LeNetV3 结构

和 ResNet18 相比，将首层的 7x7 卷积改成 3x3，删去了池化层，以更加适应

Cifar-10 数据集。

首先定义单个残差块，以实现图 2-8 的效果。

```python
class BasicBlock(nn.Module):
    expansion = 1
    def __init__(self, in_planes, planes, stride=1):
        super(BasicBlock, self).__init__()
        self.conv1 = nn.Conv2d(
            in_planes, planes, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3,
                               stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != self.expansion*planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, self.expansion*planes,
                          kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(self.expansion*planes)
            )
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

图 2-18　残差块定义

然后定义残差网络：

```python
class ResNet(nn.Module):
    def __init__(self, block, num_blocks, num_classes=10):
        super(ResNet, self).__init__()
        self.in_planes = 64
        # 修改后的初始层适应32x32输入
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3,
                               stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
        self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.linear = nn.Linear(512 * block.expansion, num_classes)
        # 权重初始化
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
    def _make_layer(self, block, planes, num_blocks, stride):
        strides = [stride] + [1]*(num_blocks-1)
        layers = []
        for stride in strides:
            layers.append(block(self.in_planes, planes, stride))
            self.in_planes = planes * block.expansion
        return nn.Sequential(*layers)
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))  # [B, 64, 32, 32]
        out = self.layer1(out)                 # [B, 64, 32, 32]
        out = self.layer2(out)                 # [B, 128, 16, 16]
        out = self.layer3(out)                 # [B, 256, 8, 8]
        out = self.layer4(out)                 # [B, 512, 4, 4]
        out = self.avg_pool(out)               # [B, 512, 1, 1]
        out = out.view(out.size(0), -1)        # [B, 512]
        out = self.linear(out)                 # [B, 10]
        return out
```

图 2-19　残差网络定义

然后对于改进型 LeNetV3，则直接使用 ResNet(BasicBlock, [2, 2, 2, 2])来进行定义即可。

使用该模型进行训练，得到的结果如图 2-10 所示，和之前的结果相比，准确率提高到了 93.5%，提升了约 1%。

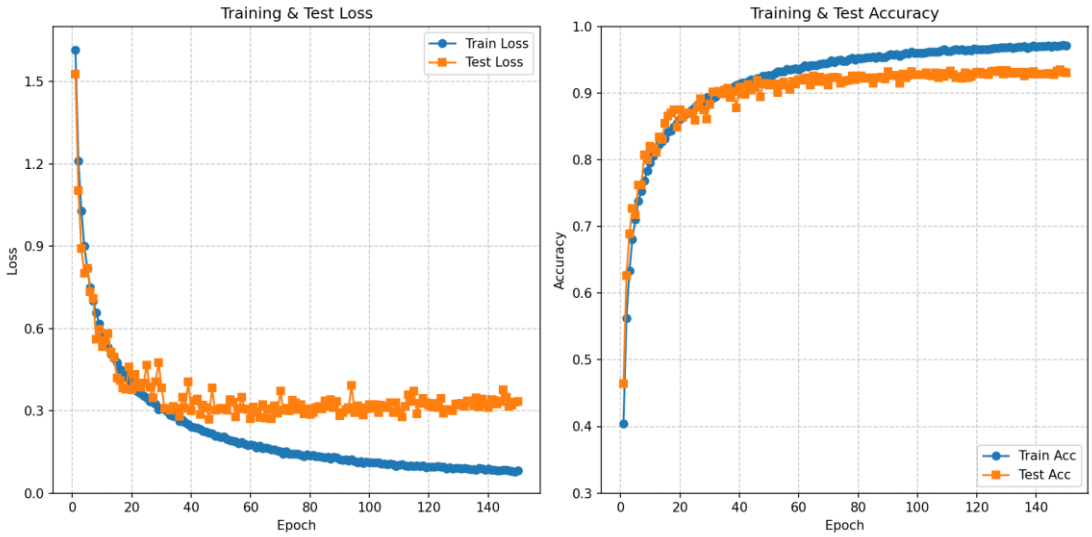

图 2-10 LeNetV3 表现

2.4 稠密连接

DenseNet 模型是 ResNet 模型的扩展，它建立的是前面所有层与后面层的密集连接（dense connection），它的名称也是由此而来。

稠密网络主要由 2 部分构成：稠密块（dense block）和过渡层（transition layer）。前者定义如何连接输入和输出，而后者则控制通道数量，使其不会太复杂。

每个稠密块内部如图 2-11 所示，整个稠密网络如图 2-12 所示。

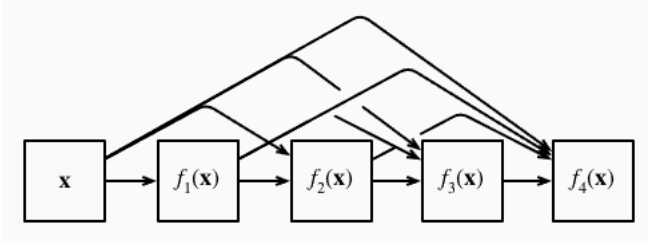

图 2-11　稠密块

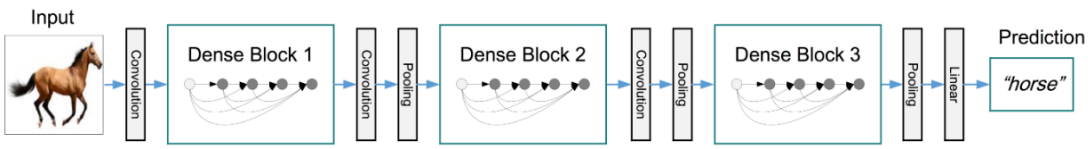

图 2-12　稠密网络

基于这个思想，可以继续调整我们的模型，定义一个含 4 个稠密块，每个块 8 层的神经网络，作为 LeNetV4 的改进型。

首先定义基础的 DenseNet 单元，包含两个卷积层和一个特征拼接操作。

```python
class DenseLayer(nn.Module):
    def __init__(self, in_channels, growth_rate):
        super(DenseLayer, self).__init__()
        self.bn1 = nn.BatchNorm2d(in_channels)
        self.conv1 = nn.Conv2d(in_channels, 4*growth_rate, kernel_size=1, bias=False)
        self.bn2 = nn.BatchNorm2d(4*growth_rate)
        self.conv2 = nn.Conv2d(4*growth_rate, growth_rate, kernel_size=3, padding=1, bias=False)
    def forward(self, x):
        out = self.conv1(F.relu(self.bn1(x)))
        out = self.conv2(F.relu(self.bn2(out)))
        return torch.cat([x, out], 1)
```

图 2-20DenseLayer 定义

然后定义密集块，由多个 DenseLayer 组成，每个新层的输入通道数是 in_channels + 已添加层数 * growth_rate，经过 num_layers 次拼接后，输出的通道数为 in_channels + num_layers * growth_rate。

```python
class DenseBlock(nn.Module):
    def __init__(self, num_layers, in_channels, growth_rate):
        super(DenseBlock, self).__init__()
        self.layers = nn.ModuleList()
        for i in range(num_layers):
            self.layers.append(DenseLayer(in_channels + i*growth_rate, growth_rate))
    def forward(self, x):
        for layer in self.layers:
            x = layer(x)
        return x
```

图 2-21　DenseBlock 定义

之后定义过渡层，连接两个 DenseBlock，通过 1x1 卷积压缩通道数，并用平均池化下采样。

```python
class Transition(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(Transition, self).__init__()
        self.bn = nn.BatchNorm2d(in_channels)
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False)
        self.pool = nn.AvgPool2d(2, stride=2)
    def forward(self, x):
        x = F.relu(self.bn(x))
        x = self.conv(x)
        x = self.pool(x)
        return x
```

图 2-22　Transition 定义

然后定义完整的 DenseNet 模型，首先定义初始的 3*3 卷积，提取低级特征。之后交替堆叠 DenseBlock 和 Transition，每个 DenseBlock 后接一个 Transition，Transition 将通道数减半，并用平均池化缩小特征图尺寸。最后是全局平均池化与全连接层。

```python
class DenseNet(nn.Module):
    def __init__(self, growth_rate=12, block_config=(6,6,6), num_classes=10):
        super(DenseNet, self).__init__()
        # Initial convolution
        self.features = nn.Sequential(
            nn.Conv2d(3, 2*growth_rate, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(2*growth_rate),
            nn.ReLU(inplace=True)
        )

        num_channels = 2*growth_rate
        # Dense blocks
        self.blocks = nn.ModuleList()
        for i, num_layers in enumerate(block_config):
            block = DenseBlock(num_layers, num_channels, growth_rate)
            self.blocks.append(block)
            num_channels += num_layers * growth_rate

            if i != len(block_config)-1:
                trans = Transition(num_channels, num_channels // 2)
                self.blocks.append(trans)
                num_channels = num_channels // 2
        # Final layers
        self.final_bn = nn.BatchNorm2d(num_channels)
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Linear(num_channels, num_classes)
        # Weight initialization
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
    def forward(self, x):
        x = self.features(x)
        for block in self.blocks:
            x = block(x)
        x = F.relu(self.final_bn(x))
        x = self.avgpool(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

图 2-23　DenseNet 定义

使用该模型进行训练，得到的结果如图 2-13，测试集准确率可达 93.11%。

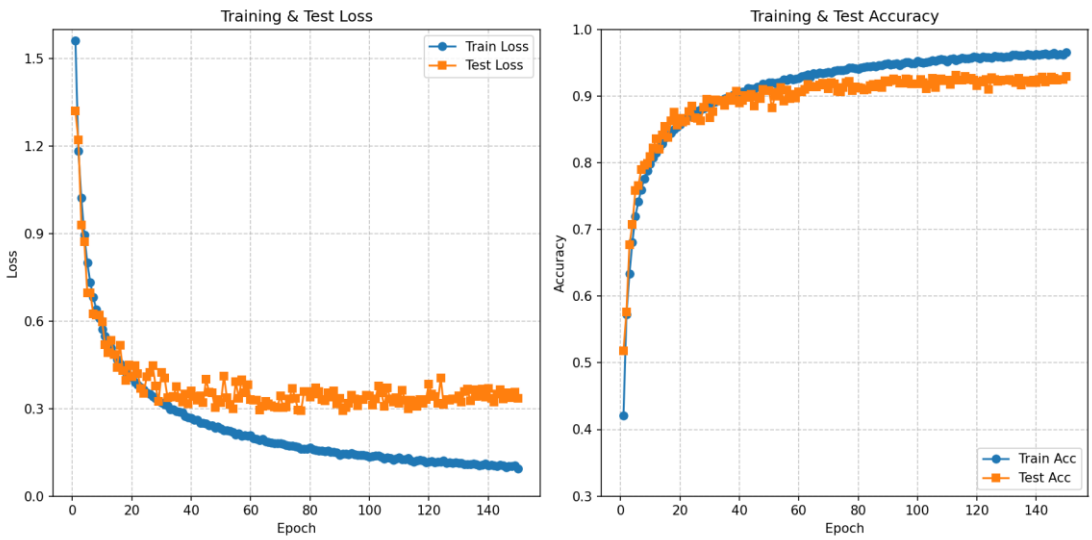

图 2-13　LeNetV4 表现

2.5　网中网

在 2014 年的 ImageNet 图像识别挑战赛中，一个名叫 GoogLeNet 的网络大

放异彩。 GoogLeNet 吸收了 NiN 中串联网络的思想，并在此基础上做了改进。这篇论文的一个重点是解决了什么样大小的卷积核最合适的问题。本文的一个观点是，有时使用不同大小的卷积核组合是有利的。在本节，将基于此网络实现 LeNetV5 改进型。

在 GoogLeNet 中，基本的卷积块被称为 Inception 块，由四条并行路径组成。

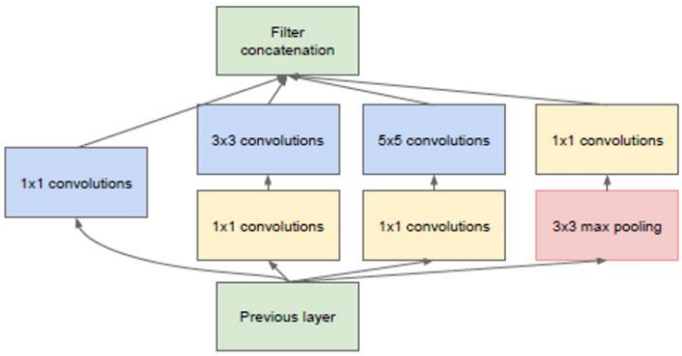

图 2-24　Inception 块结构

GoogLeNet 一共使用 9 个 Inception 块和全局平均汇聚层，其中也包括了辅助分类器，用于辅助收敛。

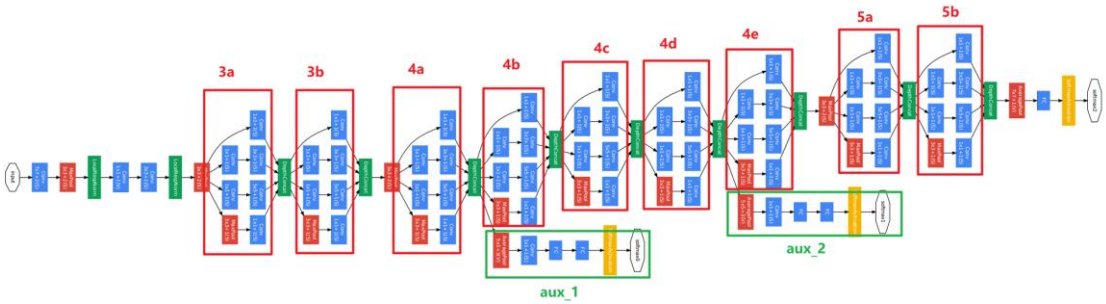

图 2-25　GoogLeNet 整体结构

基于此，我实现的 LeNetV5 改进型中，使用相似的分支并引入了残差来避免梯度消失，结构如图 2-26，包括 1×1 卷积，1×1 → 3×3 卷积，1×1 → 3×3 → 3×3 卷积，平均池化 → 1×1 卷积四个分支，然后所有分支的输出在通道维度拼接后与残差相加。

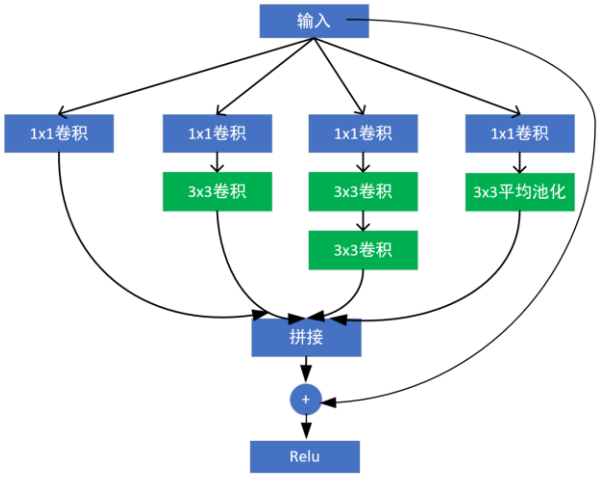

图 2-26　Inception-ResNet 模块

使用如下的代码可以定义这个模块。

```python
class InceptionResBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.branch1 = nn.Sequential(
            nn.Conv2d(in_channels, out_channels//4, 1, stride=stride, padding=0),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True)
        )

        self.branch2 = nn.Sequential(
            nn.Conv2d(in_channels, out_channels//4, 1, padding=0),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels//4, out_channels//4, 3, stride=stride, padding=1),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True)
        )

        self.branch3 = nn.Sequential(
            nn.Conv2d(in_channels, out_channels//4, 1, padding=0),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels//4, out_channels//4, 3, padding=1),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels//4, out_channels//4, 3, stride=stride, padding=1),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True)
        )

        self.branch4 = nn.Sequential(
            nn.AvgPool2d(3, stride=stride, padding=1),
            nn.Conv2d(in_channels, out_channels//4, 1, padding=0),
            nn.BatchNorm2d(out_channels//4),
            nn.ReLU(inplace=True)
        )

        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride=stride),
                nn.BatchNorm2d(out_channels)
            )
    def forward(self, x):
        identity = self.shortcut(x)
        b1 = self.branch1(x)
        b2 = self.branch2(x)
        b3 = self.branch3(x)
        b4 = self.branch4(x)
        out = torch.cat([b1, b2, b3, b4], 1)
        out += identity
        return F.relu(out)
```

图 2-27　InceptionResBlock 定义

整体的网络结构如图 2-28 所示，包括初始卷积层，之后是 4 个阶段，每个

阶段包含 3 个 InceptionRes 模块，随阶段增加，通道数增加，阶段内部的 InceptionRes 通道数不变，最后输出类别。

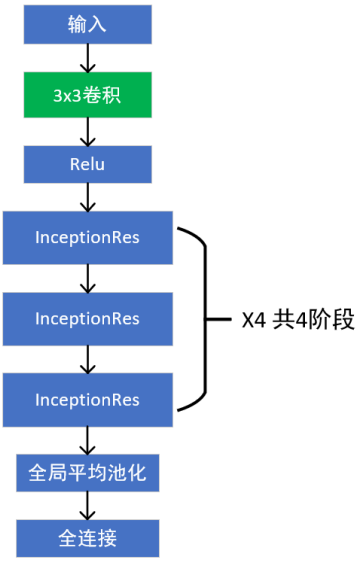

图 2-28　LeNetV5 结构

使用下面的代码可以定义这个网络。

```python
class LeNet5_v5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)

        # Stage layers
        self.layer1 = self._make_layer(64, 64, 3, stride=1)
        self.layer2 = self._make_layer(64, 128, 3, stride=2)
        self.layer3 = self._make_layer(128, 256, 3, stride=2)
        self.layer4 = self._make_layer(256, 512, 3, stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1,1))
        self.dropout = nn.Dropout(0.5)
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, in_channels, out_channels, num_blocks, stride):
        layers = [InceptionResBlock(in_channels, out_channels, stride)]
        for _ in range(1, num_blocks):
            layers.append(InceptionResBlock(out_channels, out_channels, stride=1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)

        x = self.avgpool(x)
        x = x.view(x.size(0), -1)
        x = self.dropout(x)
        x = self.fc(x)
        return x
```

图 2-29　LeNetV5 定义

使用 LeNetV5 模型进行训练，得到的结果如图 2-30 所示，准确率最高可达 93.17%。

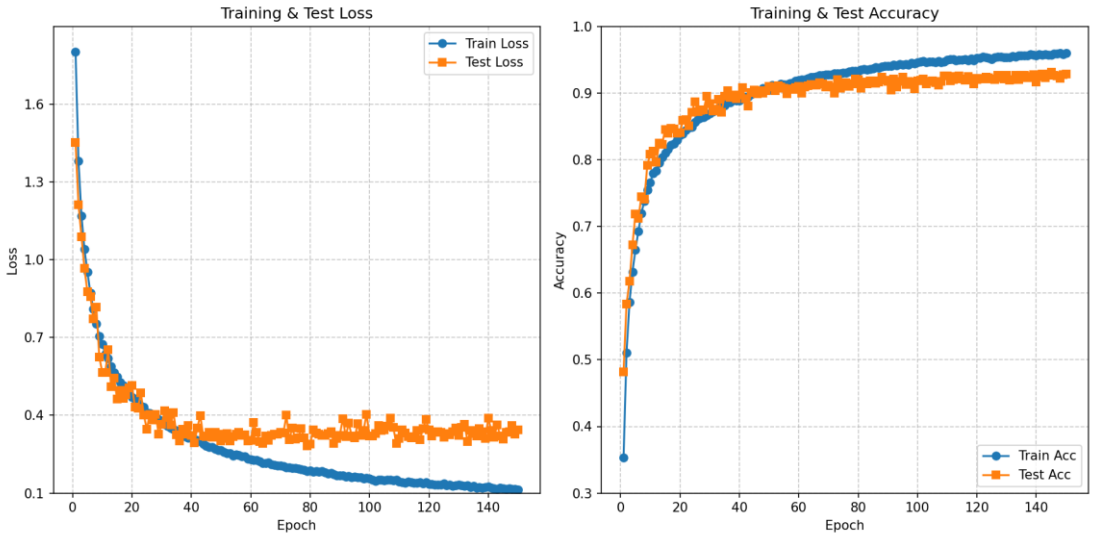

图 2-30　LeNetV5 表现

三、总结与体会

通过本次实验，使我了解到基本的卷积神经网络通过 pytorch 的实现方法，包括如何定义各个层，如何进行训练等。而且通过对 LeNet 的改进，也了解到不同架构的神经网络，以及他们各自的优缺点，改进了什么等等。

在实验过程中，也进行了很多次尝试，包括调整优化器，使用 SGD 优化器，进行了小部分的调参和训练，但是，最后发现效果没有想象的好，因此没有写入实验报告。但是，这也让我懂得，对于神经网络的调整，在准确率到达一个水平的时候，就很难再继续改进提高性能了，要么需要非常精细的调整学习率，要么就需要更深的神经网络。

我认为，如果想要继续用简单的办法提高性能，就要使用更大的数据集，模型的复杂度与数据集的复杂度有紧密的关系，也有可能实验中的架构对 CIFAR-10 数据集的特征挖掘已接近饱和。